MEMMUX: Runtime Verification and Honest Resource Attribution for Fleets of Parallel Coding Agents

Anonymous Author(s)
Affiliation
Address
email

Abstract

Developers increasingly run not one coding agent but a fleet of them side by side on one workstation. The tools they reach for, terminal multiplexers like `tmux` and a new generation of agent managers, were built to arrange windows, not to govern memory. When ten agents each spawn language servers, test runners, and browsers, no standard tool in that stack can say how much memory belongs to which agent, confirm that a terminated agent's descendants are gone, notice a child that has escaped its agent, or keep the machine off the swap cliff when an out-of-memory (OOM) kill would silently discard uncommitted work. We treat these as runtime-verification problems. An agent-hosting substrate should continuously emit observable signals that an operator or auditor can check while agents run. We present MEMMUX, an unprivileged local runtime—no kernel control groups—that turns resource governance into a set of checkable signals (per-agent attribution, complete reclamation, escaped-process visibility, bounded footprint under overcommit, and monitoring overhead) and a claims-disciplined benchmark that measures each one against `tmux` and a raw-process baseline on identical workloads, with a purpose-built agent multiplexer where it could be driven. Under a binding memory budget on a Linux host, MEMMUX keeps the fleet under budget (7500 MiB) with zero swap by admitting a subset and reclaiming under pressure. The ungoverned tools instead run every agent, pin the machine at its RAM ceiling (2× over budget), and spill about 2 GiB into swap. MEMMUX reclaims 100% of a terminated agent's process subtree where the raw baseline strands half of it (ten orphaned processes, 303 MiB; `tmux` matches us here, the raw baseline does not), and it is the only launcher we compare against that surfaces an escaped child, detecting all 10 of 10 injected escapes. The cost is real and we report it: the always-on 1 Hz monitoring loop runs near 0.6% CPU for one agent but reaches 2.7% at ten, above our 2% target. Running the same harness on real Claude Code sessions shows that 100% attribution and low overhead carry over to live, multi-process agent trees. We release the engine, the benchmark, and a one-command reproducer.

1 Introduction

A year ago, running an autonomous coding agent meant babysitting a single session. Today it is normal to fan a task out across several agents at once—one refactoring, one writing tests, one chasing a flaky build—and to watch them in a grid of panes. The agents themselves have improved quickly [4, 10, 12, 13]; the substrate they run on has not. Developers arrange these fleets with termi-

Submitted to 40th Conference on Neural Information Processing Systems (NeurIPS 2026). Do not distribute.

nal multiplexers such as `tmux` [8] or with purpose-built agent managers, and both were designed to place windows on a screen, not to answer questions about memory.

Those questions matter because a coding agent is not a single process. It is a tree: the model's CLI, a shell, language servers, test and build workers, sometimes a headless browser. A handful of such trees can exhaust a laptop's RAM. When they do, the operating system's response—the Linux OOM killer, or swap thrashing—is blunt and invisible: a process is killed mid-write, or the whole machine grinds, and the agent's uncommitted edits are simply gone. Nothing in the stack was accountable for that memory in the first place. `tmux` does not know that pane 7's language server belongs to the same logical agent as pane 3's test runner; it cannot tell you that a double-forked child has escaped supervision; and it has no budget it is trying to keep.

We think the right lens for this is *runtime verification* [1]: rather than prove properties of the substrate ahead of time, instrument it so the properties it claims can be *checked while it runs*, and surface any violation rather than hide it. Cluster schedulers made an analogous move a decade ago—Borg combined admission control, over-commitment, and process-level isolation, and treated monitoring as a first-class output [11]. We bring that stance down to a single developer machine and to the specific failure modes of agent fleets.

This reframes an infrastructure problem as a verification one, on an axis the agent-evaluation literature largely leaves open. Most work on verifying agents asks whether an agent's *outputs* are trustworthy: did it resolve the issue, is the reward genuine, is the behaviour aligned. We ask a different question, one that matters more once agents run in parallel and unattended. Does the *substrate that hosts them* uphold operational guarantees an operator can check as the fleet runs: accurate accounting, complete reclamation, and no silently lost work? These are always-on signals over the live execution, not one-shot scores. A fleet whose host cannot answer them is unverified, however capable each agent is. This is a precondition for verifying agents rather than a detour around it: an agent's observed behaviour is attributable and reproducible only if the host did not silently swap it, starve it, or OOM-kill it mid-task. A trace collected on an unverified substrate cannot separate a model that failed from a machine that did.

This paper contributes three things. First (Section 4), **five verification signals for agent fleets**: per-agent attribution, complete reclamation on teardown, escaped-process visibility, bounded footprint (with preserved work) under overcommit, and monitoring overhead. Four are runtime properties; the fifth is the monitor's own cost. Each has a stated threshold. Second (Section 3), **MemMux**, a local runtime that observes the signals and enforces the governance properties. Third (also Section 4), **a claims-disciplined benchmark** that drives `tmux`, an agent multiplexer, and a raw baseline through *identical* deterministic workloads, tags every process as agent or manager overhead, and never reports a number it did not measure. A competitor that cannot be driven fairly is recorded as skipped, not estimated. We evaluate across two Linux hosts; on the smaller one a binding-budget experiment separates MemMux from the ungoverned tools by keeping a twelve-agent stress workload under budget and off swap (Section 5); Section 6 states the limits: mostly synthetic workloads, fleets no larger than twelve, and an overhead target not yet met.

2 Background and Related Work

Language-model agents. Interleaving reasoning with tool use [13] and adding self-reflection [10] turned language models into agents that take long sequences of actions. Coding is the canonical proving ground: SWE-bench frames the task as resolving real GitHub issues against a repository [4], and SWE-agent shows a well-designed agent-computer interface matters as much as the model [12]. Multi-agent settings, first studied for open-ended behaviour [9], are now a practical reality on developer machines. This work is orthogonal to the agents: we govern whatever they spawn.

Terminal multiplexers. `tmux` [8] is the default way developers run many long-lived sessions at once. It is excellent at persistent panes and detach/reattach, and it is the baseline any "run many agents" tool must beat. But a pane is just a process group under a server, with no notion of a logical agent, a memory budget, or a process that has wandered out of its subtree.

Resource management. Cluster managers solved fleet-scale governance long ago: Borg with admission control and over-commitment [11], Mesos with two-level scheduling [3], and Kubernetes

as the open-source lineage [2]. The enforcement primitives they use—Linux control groups [5] and the OOM killer [6]—live in every modern kernel. MEMMUX borrows the ideas (reserve, admit, reclaim before you thrash) but targets a single un-orchestrated laptop, and it deliberately avoids hard cgroup containment in favor of signal-based supervision. This is an explicit trade-off: cgroups give hard, kernel-enforced limits but need privilege and per-machine setup, whereas signal-based supervision runs unprivileged and portably (macOS included) at the cost of best-effort rather than hard containment—a child can escape between samples, and one that escapes before it is ever observed is not caught at all (H3; the five signals are defined in §4). We take that trade knowingly. A kernel cgroup bounds a footprint harder than 1 Hz supervision can, and where privilege and delegation are available it is the stronger enforcement mechanism. What it does not do is decline to start work that will not fit, or preserve uncommitted work before destroying a process; and it is unavailable unprivileged and absent on macOS. A quantified head-to-head against a `systemd`-scope baseline under `MemoryMax` would separate what MEMMUX contributes from what the kernel already provides, and is the clearest next experiment.

Memory accounting. Summing resident set size (RSS) over a process tree double-counts shared pages. The proportional set size (PSS) exposed through `/proc/<pid>/smaps_rollup` divides each shared page by its sharers and is the fair per-process figure [7]; macOS offers `phys_footprint` as its analogue. MEMMUX attributes by PSS (RSS fallback) so a task's number reflects memory it is responsible for.

Runtime verification. Rather than verify a system statically, runtime verification instruments it and checks a specification against the execution trace, reacting to violations [1]. Our first four signals are such monitors: cheap, always-on checks with explicit thresholds whose failures are events, not silence; the fifth quantifies their runtime cost.

3 The Design of MEMMUX

MEMMUX is a Rust workspace: a daemon (`memmuxd`) that owns state and does the governing, and a terminal client that attaches over a Unix domain socket (UDS). A *task* is a durable, logical agent; a *runtime* is one physical launch of a provider inside a pseudo-terminal. The design is organized around making each signal checkable (Figure 1). Enforcement throughout is signal-based and unprivileged: MEMMUX uses no kernel control groups, so every budget below is held by admission and reclamation decisions rather than by kernel containment.

Attribution. Once per second the daemon snapshots the host process tree and, for each running task's provider root, sums the PSS of the root and all descendants. That per-task figure feeds three consumers: the budget, the client's task view, and the escaped-process check (which uses the pids recorded under each root). Attribution is scoped to processes MEMMUX launched, so unrelated host activity never pollutes a task's number.

Recursive reclamation. When a task is terminated (by the operator or because its provider exited), MEMMUX reaps the *entire* owned subtree. Order matters: it signals the subtree while the root is still alive, because once the root dies its children reparent to `init` and the tree that identifies them is lost. For a provider that already exited, MEMMUX reaps from the pids it recorded while the task ran. The target is at least 99.5% of owned descendants gone within ten seconds.

Escaped-process reconciliation. A child that double-forks and reparents to `init` has escaped its agent. This is the classic way a leak hides. Each sample, MEMMUX reconciles the pids seen under a task against the live tree; a once-owned pid now living outside every task subtree is flagged *escaped* and surfaced as an event. Reconciliation is scoped to the managed set, so the signal is a real "we lost track of a process we started," not host noise.

Admission and the pressure ladder. The daemon holds a memory budget (physical RAM minus reserves for the OS, interactive apps, and a swap-safety margin). Starting a task reserves its predicted peak; if the reservation would exceed the budget the task stays queued with a visible reason rather than over-subscribing the machine. When live usage climbs anyway, a graduated *pressure ladder* responds before the machine thrashes: reclaim escaped children, hibernate the lowest-priority idle

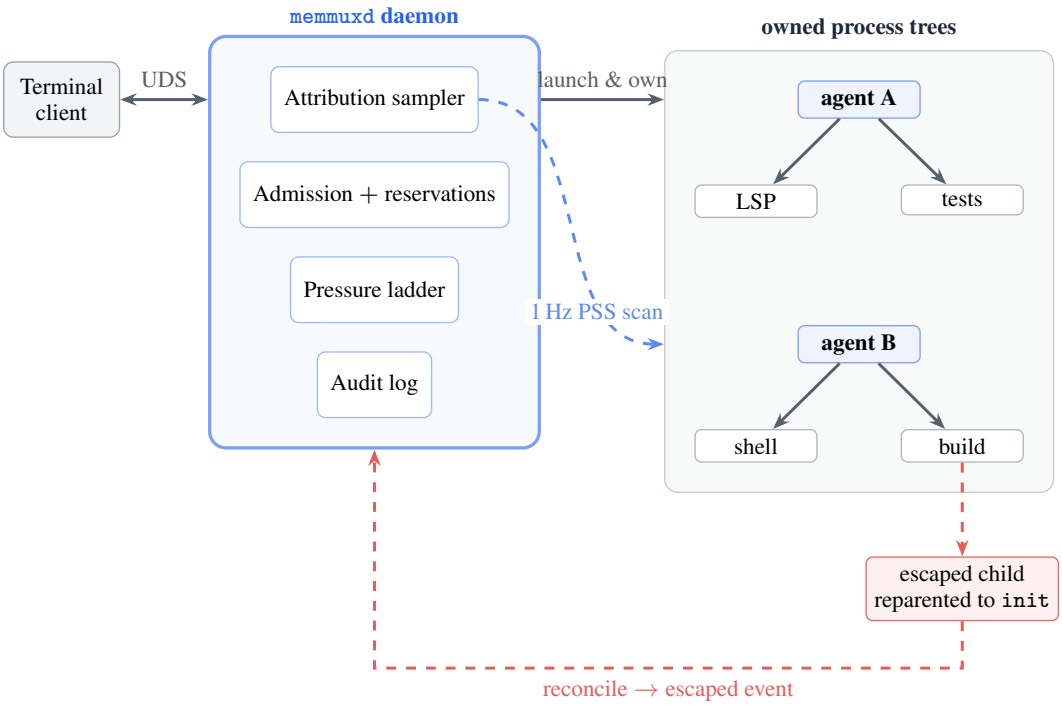


Figure 1: MEMMUX architecture. A terminal client drives the `memmuxd` daemon over a Unix domain socket. The daemon launches and *owns* each agent's process tree; a 1 Hz sampler sums proportional memory (PSS) across every owned subtree (H1), reconciliation flags any child that double-forks out to `init` (H3), and admission, the pressure ladder, and an audit log turn the budget into checkable, logged decisions (H4).

task, recycle the largest provider, and, at the top of the ladder, terminate the lowest-priority tree, but only after persisting a checkpoint of the victim's git state. Preserving work always precedes destroying a process. Every governance action writes an audit decision with its reason and evidence, so the system is auditable, not just automatic. These decisions are the only thing holding the budget—there is no cgroup beneath it—which is what makes bounded footprint a measured property here rather than a kernel-guaranteed one.

4 Verification Signals and Methodology

We frame governance as runtime monitors over the fleet's execution. The observed *trace* is the sequence of once-per-second process-tree samples and lifecycle events; each signal is a *monitor* over that trace with an explicit threshold (Table 1), and a breach is surfaced as a violation event rather than silently tolerated. Four are properties of the running system (H1–H4); H5 is the *cost* of monitoring, not an invariant, and H4 bundles three sub-properties—admission, bounded footprint, and preservation of uncommitted work—which we report separately below as **H4a** (admission and bounded footprint) and **H4b** (preservation of uncommitted work ahead of a destructive termination); the generated run reports use the same labels. MEMMUX both *observes* these signals and *enforces* them (admission, reclamation); the baselines can only be observed. These are empirical monitors with measured pass/fail behaviour, not machine-checked proofs, so we claim the signals are checkable and checked, not that the engine is sound and complete in the formal sense. A benchmark harness drives every launcher through identical, deterministic workloads and evaluates the monitors from one external process sampler—an *oracle* using the daemon's own accounting code, pointed at whatever root pids a launcher reports, so measurement is product-agnostic.

Launchers. Each launcher starts N identical agents and reports the pids that constitute the fleet: a *raw* baseline (spawn the stub directly), `tmux` (one pane per agent on an isolated server), and

	Signal	Monitor / threshold
H1	Attribution	native signal maps $\geq$ 95% of a launched tree's proportional memory to its owning task
H2	Cleanup	$\geq$ 99.5% of an owned subtree gone $\leq$ 10 s after the launcher's normal teardown
H3	Escaped visibility	every previously observed process that escapes and remains live through the next sample is surfaced as an event
H4	Bounded / no-loss	peak footprint $\leq 1.1\times$ budget under overcommit; uncommitted work checkpointed before destructive termination
H5	Overhead	continuous monitoring costs $\leq 2\%$ CPU at $N{=}10$

Table 1: The five runtime-verification signals and their pass thresholds.

MEMMUX (create N tasks through the real daemon). A fourth, the `herdr` agent multiplexer, is driven through its socket API where installed; on our stock Linux reference host it was absent and is reported as skipped rather than guessed, as are `dmux` (which requires a controlling terminal and cannot be driven headlessly), `cmux`, and `agentmux`.

Workload. A deterministic stub agent replays a recording of allocate / spawn-child / emit / sleep steps, touching every page it allocates so its resident memory is real. We use a stub rather than a live model on purpose: runs are then reproducible and identical across launchers, at the cost of realism (Section 6). A *hold* scenario keeps N agents, each with a live child, resident together for the measurement window. Without it, short workloads finish before a fleet can be observed, an error we hit early and corrected.

Claims discipline. The harness records each launcher's version, the measurement date, and the host; it tags every sampled process as agent or manager overhead so the two are reported separately; and it emits a number only for a monitor it actually measured. A launcher that cannot be driven contributes "unsupported," never a fabricated zero. These rules are part of the contribution: there is no accepted methodology for comparing agent-hosting tools, and an unfair comparison is worse than none. One point needs care. Because the oracle can attribute *any* launcher's tree after the fact, per-agent attribution is *measurable* for all of them, and the real-agent run reports 100% for every launcher (Table 4). H1 as a signal asks the stricter question of whether a launcher exposes that attribution *natively and continuously* to an operator. Only MEMMUX does; `tmux` and the raw baseline surface no such figure themselves. Table 2 marks that native capability, not the oracle's coverage.

5 Evaluation

Setup. Signals H1–H3 and H5 run on an AWS `m7i.2xlarge` reference host (8 vCPUs, Intel Xeon Platinum 8488C, ≈31 GiB RAM, Ubuntu 24.04), sweeping $N \in \{1, 5, 10\}$ over two trials. The bounded-footprint claim (H4) needs *real* memory pressure, so we run it on a deliberately smaller 15.3 GiB host (`m7i.xlarge`, 4 vCPUs, 12 GiB swap) with a calibrated synthetic stress footprint: each agent allocates 1400 MiB, matching the reservation prior of MEMMUX's default *Standard* task class, so admission is well-calibrated (reservation ≈ actual) rather than over-reserving; each resulting process tree holds ∼1.5 GiB, and $N{=}12$ of them (∼18 GiB) genuinely overcommit the machine. The H1–H3/H5 sweeps use two trials as reproducibility checks and report their means without population-level confidence claims. The H4 run uses five trials; its archived report gives mean ± 95% confidence intervals for footprint, while Table 3 rounds the means and reports observed peak swap. Signals H1–H3 and H5 were measured on MEMMUX 0.8.0, H4 on 0.9.0 (the release that added the configurable stress footprint), and the H4b checkpoint demonstration on 0.9.1; these differ in benchmark tooling and in the checkpoint's target directory rather than in the governance paths under test, but no single build has been measured across all five signals in one sweep. A macOS cross-check (Apple M4 Max, 36 GiB) that also exercised `herdr` agreed qualitatively: like the other ungoverned managers, `herdr` reaped its subtrees on teardown but saw no escapes and grew with the fleet. Table 2 summarizes the head-to-head; the subsections that follow are the evidence behind each cell.

Launcher	Attrib.	Cleanup	Escaped	Peak footprint (MiB)	Swap (MiB)	Ckpt.
raw baseline	×	50%	×	15075	1882	×
`tmux` 3.4	×	**100%**	×	15347	2165	×
MEMMUX[‡]	✓	**100%**	✓ 10/10	**7479**	**0**	✓[†]

Table 2: Head-to-head on the signals (Linux; see §5 for the per-signal hosts). MEMMUX is the only launcher that natively attributes per-agent memory (H1), surfaces escaped processes (H3), holds the fleet *under* a binding memory budget with zero swap (H4), and can checkpoint dirty work before destructive termination. It *matches* `tmux` on teardown cleanup (H2). Footprint/swap are peaks under a binding 7500 MiB budget on a 15.3 GiB host (N=12); the ungoverned tools overcommit to the RAM ceiling and spill ∼2 GiB into swap. Attribution is the *native* capability, not the oracle's coverage (§4); MEMMUX also pays a monitoring cost the baselines do not (H5, §5.4). [†]Checkpoint-before-destructive-termination is exercised end-to-end on a single task (§5.1); automated resume from that checkpoint is not. [‡]H1–H3/H5 use MEMMUX 0.8.0; H4 and the live forced-kill run use 0.9.0; the H4b demonstration uses 0.9.1. Each cell is the headline result for that signal.

Launcher	Peak (MiB)	Steady (MiB)	Peak swap (MiB)
raw baseline	15075	15069	1882
`tmux` 3.4	15347	15069	2165
MEMMUX 0.9.0	**7479**	**4512**	**0**

Table 3: Binding-budget *overcommit* (N=12 agent trees, ∼1.5 GiB each, 15.3 GiB host, five trials). MEMMUX's 7500 MiB budget is half the ∼15 GiB the ungoverned fleet uses; MEMMUX holds peak footprint under budget with zero swap, while raw and `tmux` overcommit the machine 2× and spill ∼2 GiB into swap.

5.1 H4a and H4b: bounded footprint and preserved work under overcommit

We make the budget *binding* using the calibrated stress workload of §5: N=12 agent trees (∼18 GiB) against a 15.3 GiB host, with MEMMUX given a 7500 MiB budget—roughly half what the fleet wants. Table 3 reports peak total footprint and peak swap over five trials. The ungoverned launchers run all twelve agents: their resident set pins the machine at its ∼15 GiB RAM ceiling—2× over the budget—and spills 1.8–2.1 GiB into swap. MEMMUX instead governs, holding peak footprint at 7479 MiB (*under* budget), settling to 4512 MiB steady, with **zero** swap. The margin is narrow—21 MiB against a 7500 MiB budget—and it is a *sampled* peak: a 1 Hz observer can miss a brief excursion between two ticks, so the figure bounds what was observed rather than what was reached. A kernel-provided high-water mark would settle that directly and is the obvious next measurement. The zero-swap result is less exposed to the same objection, since swapped pages persist across ticks rather than appearing only between them. The run recorded seven deferred starts and twelve reclamation events in total; these are event counts rather than unique tasks.

The trade-off is throughput. MEMMUX admits at most about five agents and can settle lower as pressure acts, so enforcing the budget means not running every agent at once. That is what a budget is for: the ungoverned tools run all twelve only by driving the machine into swap, where an OOM kill can strike. We do not measure the resulting completion time. Admission converts a memory overrun into queueing delay, and quantifying that exchange—makespan under governance against the wall-clock cost of an OOM kill and the work it destroys—is the comparison a user deciding whether to adopt governance would want, and we do not yet provide it. We do, however, address one half of that cost directly: the work an OOM kill destroys.

H4b: preserved work. Before the pressure ladder's *emergency* stage destructively terminates a victim, it first captures a durable checkpoint (git HEAD, dirty manifest, and patch hash). The twelve-agent run above never reached emergency termination—its reclamations were hibernate/recycle actions—so we exercised that path directly. Driving a single task with a dirty worktree past a binding budget until the ladder reaches *emergency* (MEMMUX 0.9.1, one trial, macOS), the daemon writes a checkpoint carrying the victim's real patch hash with its dirty flag set, and only then terminates the tree, reclaiming 100% of it. The recorded hash matches the one the independent worktree-retention path computes for the same tree, so the two agree on what was preserved. Recovery today means that retained worktree together with its checkpoint: automated *resume* is wired for hiber-

nated tasks, not for emergency-terminated ones, so this is preservation rather than one-command restoration. Ungoverned launchers have no checkpoint mechanism at all.

5.2 H2: complete cleanup on teardown

Each launcher starts a fleet whose agents each hold a memory-bearing child, then is torn down exactly as it normally would be (raw: kill the roots; `tmux`: `kill-server`; MEMMUX: terminate each task). Ten seconds later we count survivors from the owned set. The raw baseline strands every child it spawned—its teardown kills only the roots, orphaning their children—and the leak grows with the fleet: one process and 30.6 MiB at $N{=}1$, ten processes and 303 MiB at $N{=}10$ (50% reclaimed at every size). `tmux` and MEMMUX both reap their full subtree at every size (0 leaked, 100% reclaimed). On *explicit* teardown, then, cleanup separates MEMMUX from the raw baseline but not from a real multiplexer: `tmux` reaps its subtrees too. This is the kind of result the claims discipline is meant to surface.

5.3 H3: escaped-process visibility

We inject escapes: a stub forks an intermediate that spawns a memory-holding grandchild and then exits, reparenting the grandchild to `init` while it stays alive. The harness independently confirms the reparenting, then asks each launcher what it detected. MEMMUX surfaces every escape—1/1, 5/5, 10/10—through events read from the daemon; the baselines have no mechanism to notice, so they are recorded as unsupported, never scored as a measured zero. This is a capability the baselines lack rather than a tuning difference; with no baseline to score against, we report it as a capability demonstration rather than a measured separation. Two limits bound what the signal covers. First, detection is scoped to processes MEMMUX has already sampled under a task: reconciliation flags a *once-owned* pid now living outside every task subtree, so a child that forks and reparents before it is ever observed falls outside the signal, however long it then survives. Second, reconciliation runs at 1 Hz, so a child that is observed and then both escapes *and* exits within a second can slip between two samples. Our injected escapes hold memory for several seconds, well past the interval. A shorter interval narrows the second window and trades detection latency for overhead (H5); observing the spawn path directly, rather than inferring ownership from periodic snapshots, would close the first gap outright and is the better fix.

5.4 H1 & H5: what the attribution scan delivers, and what it costs

We pair these two signals because they are the benefit and the price of a single mechanism: the always-on 1 Hz PSS scan produces the attribution guarantee (H1) and incurs the monitoring cost (H5). Across every run MEMMUX attributed 100% of each launched tree's proportional memory to its owning task *natively*, and the externally run oracle confirmed the figure using the shared accounting implementation (H1). This validates the product integration but does not rule out a common-mode accounting bug; an independently implemented oracle remains future work. The memory cost of governance is small: the MEMMUX daemon's own resident footprint is 4.6–5.4 MiB (growing slightly with the fleet), against 2.4 MiB for the `tmux` server and zero for the raw baseline. The CPU cost is the weak spot (H5). The MEMMUX daemon's total CPU, dominated by the always-on 1 Hz full-tree PSS scan (we did not ablate the two), is, worst case across scenarios, 0.6% at one agent, 1.5% at five, and 2.7% at ten, above our 2% target once the fleet reaches ten. The scan is $O(\text{processes})$ and currently re-reads the whole tree each tick; incremental sampling that touches only changed pids is the obvious fix and is future work. We report the miss rather than hide it.

5.5 Validation on real coding agents

To test whether the attribution and overhead results are artifacts of the stub, we ran the *same* harness against a real coding agent. Each "agent" is a headless Claude Code session (`claude -p`, v2.1.170) editing a scratch git repository, launched through the identical sampling and teardown pipeline via the benchmark's `--agent-cmd` mode. We report an $N{=}3$ hold run on the macOS host (Apple M4 Max, 36 GiB). A live Claude Code session is a multi-process, Node-based tree, so this is a strictly harder attribution target than the single-process stub.

Table 4 gives the result. MEMMUX attributes 100% of every real agent tree to its owning task—the H1 gate holds on live agents, not just the stub—and the manager cost stays small (7.2 MiB resident,

Launcher	Total steady (MiB)	Manager (MiB)	Tree attributed
raw baseline	883.6	0.0	100%
`tmux` 3.6a	883.6	2.4	100%
`herdr` 0.8.0	929.6	18.2	100%
MEMMUX 0.8.0	843.5	7.2	100%

Table 4: Real Claude Code agents (N=3, hold scenario, macOS) driven through the identical harness. Every launcher attributes the full real (Node-based) agent tree; MEMMUX's manager overhead is small. Footprint is steady-state provider+manager memory; at N=3 without an overcommit budget the totals are comparable across launchers—this run isolates *attribution and overhead on real trees*, not the overcommit footprint differentiator of Table 3.

0.012% CPU at three agents on this macOS host; the comparable Linux figure from the forced-kill run below is 1.642% at three agents, against 0.636% at one agent in the H5 sweep). Three concurrent real sessions occupy ∼0.86 GiB (∼290 MiB each), confirming that the pressure the system governs is real and substantial per agent. Attribution and overhead thus generalize from the stub to real, multi-process agent trees.

Cleanup on live agents. That hold run let every session finish, so all launchers reclaimed cleanly. To check the cleanup *mechanism* on *live* agents we ran a second experiment (Linux, `m7i.xlarge`, MEMMUX 0.9.0) in which three real Claude Code agents (v2.1.197) are *forcibly torn down mid-work*, while their `node` process trees are still resident. The raw baseline strands **three real processes and 525 MiB** of live agent memory (50% reclaimed)—killing the root `node` orphans its children to `init`, exactly as with the stub—while both `tmux` and MEMMUX reap the full subtree (0 leaked, 100%). Consistent with H2, this is a *raw*-versus-governed result, not a separation from `tmux`; its value is that the cleanup behaviour the stub isolates reproduces on real agent trees, and that the naive "run it in a terminal" baseline strands half a gigabyte of live agent memory. The escaped-process condition (H3) did not arise here, as Claude Code does not double-fork out of supervision.

6 Discussion and Limitations

Synthetic workload. The largest threat to validity: agents are a deterministic memory-ballast stub, not live models. We trade realism for reproducibility and for identical workloads across launchers, which is what makes the comparison fair. The *qualitative* guarantees—attribution, cleanup, escape visibility, admission—do not depend on the workload, but the specific megabytes for a real Claude- or GPT-driven session will differ. We take a first step in §5.5, running real Claude Code sessions through the same harness: 100% attribution and low overhead hold on live, multi-process agent trees, and forcibly killing them mid-work reproduces the raw baseline's cleanup leak (525 MiB) that the stub isolates. A larger live-agent study—more agents and longer sessions, and reproducing the H4 overcommit and H3 escape conditions on live trees—remains the most valuable next step.

Two Linux hosts, modest fleets. We evaluate H1–H3/H5 on a 31 GiB Linux host, H4 on a 15.3 GiB Linux host, and perform a macOS cross-check. Fleets are no larger than N=12; larger fleets would make the CPU-overhead question (H5) sharper.

Static reservations. MEMMUX reserves each task's *class-predicted* peak—a static prior. Our H4 run sizes agents to that prior on purpose, so admission is well-calibrated; but for an atypically small agent the prior over-reserves (deferring starts that would have fit), and for an atypically large one it under-reserves. An exponentially-weighted predictor that tightens the reservation toward each task's *observed* peak is specified but not yet wired in; the bounded-footprint guarantee is unaffected—only the admit count moves with the predictor's accuracy.

Overhead not yet met. Monitoring exceeds 2% CPU by ten agents (above); incremental sampling is the planned fix. As stated in §4, these are empirical monitors, not machine-checked proofs—checkable and checked, not formally verified.

7 Reproducibility

MEMMUX and its benchmark are open source (MIT).[1] A single command runs the full stub matrix—every launcher, scenario, and fleet size, with repeated trials—captures the host specification, and regenerates the headline tables and figures plus the raw per-sample and per-process time series; two further invocations (Appendix A) produce the binding-budget H4 and real-agent runs. Competitor versions and the measurement date are embedded in the generated report, and any launcher that could not be driven is listed with its reason. The same harness accepts a `--agent-cmd` flag that substitutes a real coding-agent command (e.g. `claude -p ...`) for the stub, so the real-agent validation of §5.5 is reproducible through the identical pipeline; that run's report and raw per-process series are archived under `paper/realagent-run/`.

Each run captures a host specification, a Markdown report, and per-cell time series (RSS, PSS, USS, swap, and page faults per process on Linux) plus the figures. We withhold the exact invocation and the repository here to preserve review anonymity; the engine, the benchmark, and the one-command reproducer—including the binding-budget H4 configuration used for Table 3—are released with the code.

8 Conclusion

Fleets of coding agents have outrun the tools we use to run them: multiplexers arrange them, but nothing governs them. We argued that memory governance for such fleets is a verification problem and built MEMMUX to make five signals checkable at runtime, with a benchmark that also reports where those signals do *not* distinguish us. The payoff is concrete. Under a binding budget MEMMUX holds the fleet under budget and *off swap* where the ungoverned tools overcommit the machine ($2\times$ over budget, $\sim$2 GiB into swap); it reclaims a terminated agent's whole subtree where the raw baseline strands half of each one; and it surfaces escaped processes that the alternatives have no mechanism to see. Its own overhead we have not yet driven under 2%. The next steps are larger and longer live-agent workloads, including live H3/H4 stress conditions, and the reproducer makes those a single command away. Checking what an agent produced is only half of verification; the other half is checking that the system running it kept its promises, and the signals and claims discipline here are a step toward that.

References

[1] Ezio Bartocci, Yliès Falcone, Adrian Francalanza, and Giles Reger. Introduction to runtime verification. In *Lectures on Runtime Verification: Introductory and Advanced Topics*, volume 10457 of *Lecture Notes in Computer Science*, pages 1–33. Springer, 2018. doi: 10.1007/978-3-319-75632-5_1.

[2] Brendan Burns, Brian Grant, David Oppenheimer, Eric Brewer, and John Wilkes. Borg, Omega, and Kubernetes. *Communications of the ACM*, 59(5):50–57, 2016. doi: 10.1145/2890784.

[3] Benjamin Hindman, Andy Konwinski, Matei Zaharia, Ali Ghodsi, Anthony D. Joseph, Randy Katz, Scott Shenker, and Ion Stoica. Mesos: A platform for fine-grained resource sharing in the data center. In *Proceedings of the 8th USENIX Conference on Networked Systems Design and Implementation (NSDI)*, 2011.

[4] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations (ICLR)*, 2024. arXiv:2310.06770.

[5] Linux Kernel Documentation. Control group v2. https://www.kernel.org/doc/html/latest/admin-guide/cgroup-v2.html, 2024. Accessed 2026-08.

[6] Linux Kernel Documentation. Memory management: Concepts overview. https://www.kernel.org/doc/html/latest/admin-guide/mm/concepts.html, 2024. Accessed 2026-08.

[7] Linux Kernel Documentation. The /proc filesystem: smaps and smaps_rollup (proportional set size). https://www.kernel.org/doc/html/latest/filesystems/proc.html, 2024. Accessed 2026-08.

[8] Nicholas Marriott and tmux contributors. tmux: a terminal multiplexer. https://github.com/tmux/tmux, 2024. Accessed 2026-08.

[1] Code, benchmark, and a one-command reproducer are open-source and will be released; the repository URL is withheld to preserve double-blind review.

[9] Joon Sung Park, Joseph C. O'Brien, Carrie J. Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. Generative agents: Interactive simulacra of human behavior. In *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology (UIST)*, 2023. arXiv:2304.03442.

[10] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. arXiv:2303.11366.

[11] Abhishek Verma, Luis Pedrosa, Madhukar Korupolu, David Oppenheimer, Eric Tune, and John Wilkes. Large-scale cluster management at Google with Borg. In *Proceedings of the Tenth European Conference on Computer Systems (EuroSys)*, 2015. doi: 10.1145/2741948.2741964.

[12] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. *arXiv preprint arXiv:2405.15793*, 2024.

[13] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023. arXiv:2210.03629.

A Reproducer invocations

Every table and figure in this paper regenerates from the commands below. Each writes `host.json`, a Markdown report, per-cell `*.jsonl` and `*.procs.jsonl` time series, and the figures into its output directory.

```
# Full matrix: every launcher, scenario, and fleet size, with repeated trials.
memmux-bench paper --out paper-out

# Binding-budget H4 (Table 3): twelve agents at ~1.4 GiB each,
# against a 7500 MiB budget on a 15.3 GiB host, five trials.
MEMMUX_BENCH_HOLD_MIB=1400 memmux-bench paper \
    --scenarios overcommit --agents-sweep 12 \
    --agent-budget-mib 7500 --trials 5 --out oc-out

# The same harness driving a real coding agent in place of the stub (Table 4).
memmux-bench run --agents 3 \
    --agent-cmd 'claude -p "..."' --out real-out
```

The default sweep is $N \in \{1, 5, 10, 20\}$ at three trials; the paper's runs override both. `MEMMUX_BENCH_HOLD_MIB` sets the per-agent resident footprint of the deterministic stub (default 120 MiB) and is what makes the H4 budget binding against a realistic agent size.

B The H4b checkpoint demonstration

This appendix gives the reproduction recipe for the H4b result in §5.1. One agent is admitted under a budget it will exceed, writes an uncommitted change into its isolated git worktree, then allocates past the budget until `hard_limit_exceeded` forces `PressureStage::Emergency`. The ordering is structural rather than timing-dependent: the stage's action list runs `PreserveGitState` ahead of `TerminateLowestPriorityTree`, so no checkpoint can be skipped by a race.

Parameters and observations (MEMMUX 0.9.1, one trial, macOS; agent budget 1600 MiB, agent footprint 2109 MiB, 132% utilization):

Observation	Value
Pressure stage reached	`Emergency` (hard limit exceeded)
Checkpoint written before termination	yes
Checkpoint `git_dirty`	`true`
Subtree reclamation	100% (2 targeted, 0 survived)
Worktree after termination	retained with the uncommitted change intact

Table 5: H4b end-to-end checkpoint demonstration (MEMMUX 0.9.1, one trial, macOS).

The recorded patch hash is compared against the one the worktree-retention path computes for the same tree; the two agreeing is what distinguishes a captured patch from the empty-diff sentinel a clean directory would produce.